GitHub Trending

每日热门项目简报

2026年11月5日 · 全球 · 全语言 · 当日榜单前五

榜单采集：北京时间 09:39。总星标、语言、许可证和源分支 SHA 由 GitHub API 在本轮采集中核验。Trending 反映当日新增关注度，不代表代码质量、安全性、成熟度或商业价值。

排名	项目	今日新增	总星标	语言 / 许可证
1	NVIDIA/auto-ontology	2,050	5,096	Python / Apache-2.0
2	max-sixty/worktrunk	531	11,053	Rust / MIT 或 Apache-2.0
3	mattpocock/skills	1,379	313,353	Shell / MIT
4	floci-io/floci	1,090	29,648	Java / MIT
5	thedotmack/claude-mem	1,172	112,880	TypeScript / Apache-2.0

星标新增来自 Trending 页面快照；总星标为 GitHub API 查询快照，采样时刻不同。

1. NVIDIA/auto-ontology - 企业数据语义与智能体上下文层

已核实事实： 仓库 README 将 Auto Ontology 描述为 Python 框架：从企业数据元数据构建或连接本体和语义模型，为指标、实体及关系提供语义上下文，并通过 MCP、CLI 和 API 暴露给智能体。仓库还包含自然语言到 SQL、Postgres/pgvector、网页 UI、FastAPI 服务、Docker Compose 与 Kubernetes Helm 配置。README 指明推理可连接 NVIDIA 托管 API 或自托管 NIM；示例部署需 Docker Compose、uv 和 NVIDIA API 密钥。默认分支 main 的采集提交为 2a24f5d34ef6b6e168a7f22d308504d3ab36a198。许可证文件及 API 均确认 Apache-2.0；README 说明当前不接受外部贡献。 当前可支持的用途： 按项目文档，可在受控数据目录上建立语义目录，让智能体查询结构化数据并返回来源信息；示例快速开始使用微软 Wide World Importers 样例数据库。文档所述用例不证明已经在任何特定生产环境部署。 潜在价值（分析推测）： 若企业已有可靠的数据目录、权限体系和经审核的定义，可减少不同系统中同名指标含义不一导致的错误查询，并为 SQL 结果附加语义和来源。准确性仍依赖元数据质量、业务人员校验、权限映射和 SQL 安全审计。 对象、入门与成本： 面向数据平台团队、分析工程师及企业 AI 工程师。可先用样例库验证目录生成和查询链路，再连接只读测试数据源。开源许可证不免除运行成本：需维护 Postgres、容器或 Kubernetes、身份与秘密管理；托管模型涉及 API 配额或费用，自托管 NIM 需要符合模型硬件要求的 GPU 和运维。真实费用取决于流量、模型、云厂商及硬件，本文未推算金额。处理企业元数据或 SQL 前，应核对数据出境、访问控制、日志和提示注入风险。 复刻： qinshanliangengineer/auto-ontology，fork parent 已核实为 NVIDIA/auto-ontology；源 main 与 fork main 比较为一致，本次未同步。

来源： 官方 README、LICENSE、GitHub 仓库元数据、采集时源分支提交。

2. max-sixty/worktrunk - Git worktree 工作流命令行工具

已核实事实： Worktrunk 是用于管理 Git worktree 的 CLI。README 介绍按分支名寻址 worktree、可配置路径模板，并列出 switch、list、merge、remove、hooks、交互式选择器、提交信息生成和 Claude Code 集成等功能。其动机之一是将不同代理或开发任务放在分离的工作目录并行进行。README 和 LICENSE 明确采用 MIT 或 Apache-2.0 双许可证；GitHub API 的标准许可证识别字段返回 NOASSERTION，因此以仓库许可证文件为准。默认分支 main 的采集提交为 3bf9166a6e3b74db0de25c2a1f9aeeae54d2873a。 当前可支持的用途： 对遵循 Git 分支和 worktree 约定的开发者，提供创建、切换、列出、合并和删除 worktree 的统一命令，降低手动维护多个检出目录的操作负担。 潜在价值（分析推测）： 若多个代理任务修改不同分支且遵守各自工作目录边界，worktree 可帮助并行开发而减少工作区互相覆盖。它不自动解决共享数据库、端口、外部状态或同一分支冲突；合并、测试和代码审查仍需要人工或 CI。 对象、入门与成本： 面向熟悉 Git 的个人和团队，可先在临时仓库试用基础 switch/list/remove，再评估 hooks。主要成本是学习路径、为每个 worktree 安装依赖产生的磁盘与构建时间，以及并行任务的审查成本；工具本身是本地 CLI，未发现必需的托管服务费用。 复刻： qinshanliangengineer/worktrunk，fork parent 已核实为 max-sixty/worktrunk；fork main 比源仓库落后 5 个提交，本次未同步。

来源： 官方 README、双许可证文件、GitHub 仓库元数据、采集时源分支提交。

3. mattpocock/skills - 面向软件工程流程的代理技能集合

已核实事实： README 将仓库定位为作者日常使用的一组小型、可编辑、可组合的工程技能，并说明可通过 skills.sh 或手动方式安装到不同代理工具中；安装后需在项目中运行 setup 技能配置 issue tracker 等工作流偏好。仓库语言分类为 Shell，主要交付物是技能说明和相关资源，而非一个独立运行的通用应用。LICENSE 文件为 MIT。默认分支 main 的采集提交为 4ae2317ddb22b046e26455e4f9c9b3aaaf62212d。 当前可支持的用途： 文档涵盖任务澄清、代码可靠性、降低代理冗长输出、逐步执行和工程设计等流程指导。可以在非关键或个人代码库中逐个启用，并检查技能规则是否与团队现有流程一致。README 中关于改善代理表现的说法属于作者主张，没有在本报告中独立复现评测。 潜在价值（分析推测）： 若团队已有明确的代码审查、测试和 issue 处理标准，将其转成代理可读取的项目技能，有望让重复性工作更一致；成效取决于团队规则是否匹配、技能是否持续维护，以及模型是否遵循指令。 对象、入门与成本： 面向使用 Claude Code、Codex 或其他兼容技能机制的工程师。建议先审阅技能文件，再在测试仓库安装单项技能并比较工作结果。主要成本是适配与维护指令、不同代理的兼容性验证和模型调用成本；安装简单不代表自动带来质量提升。技能会影响代理对代码库采取的动作，因此需阅读并审核内容后再赋予写文件或运行命令权限。 复刻： qinshanliangengineer/skills，fork parent 已核实为 mattpocock/skills；fork main 落后源仓库 495 个提交，本次未同步。

来源： 官方 README、LICENSE、GitHub 仓库元数据、采集时源分支提交。

4. floci-io/floci - 本地 AWS API 仿真环境

已核实事实： Floci README 将其描述为面向开发、测试和 CI 的开源本地 AWS 仿真器。文档称可将 AWS SDK、CLI、Terraform、CDK、OpenTofu 或测试套件指向 localhost:4566，并通过 Docker 启动；仓库提供 Web Console。README 的能力对照表列出其支持范围，因此兼容性应按具体服务和操作核验。许可证文件为 MIT。默认分支 main 的采集提交为 4abccec0c80cb15175a09316a41bb0f16772e881。 当前可支持的用途： 在本机或 CI 内，以本地端点替代真实云账号运行部分 AWS 形状的开发与测试调用。README 强调无需 AWS 账号或令牌；这是项目作者对其本地使用模式的说明。 潜在价值（分析推测）： 在受支持的 API 子集内，开发者可以减少测试阶段对云资源的依赖，并快速重复执行集成测试。它不能自动代表真实 AWS 的全部行为，尤其应关注 IAM、边缘服务特性、配额、网络、持久化和故障语义与云端的差异；发布前仍需在真实云环境做兼容性验证。 对象、入门与成本： 面向使用 AWS 兼容 SDK 和 IaC 的开发者、测试人员及 CI 维护者。建议先对照目标服务的官方兼容表，在容器中运行一个无敏感数据的最小测试。主要成本为本地 CPU、内存、磁盘、Docker 运维和适配差异；避免云调用不等于零成本，也不能据此宣称替代生产云服务。 复刻： qinshanliangengineer/floci，fork parent 已核实为 floci-io/floci；源 main 与 fork main 比较为一致，本次未同步。

来源： 官方 README、LICENSE、GitHub 仓库元数据、采集时源分支提交。

5. thedotmack/claude-mem - 为编码代理跨会话保存项目上下文

已核实事实： README 描述 claude-mem 通过代理生命周期钩子捕获会话活动、压缩观察结果，并在后续会话中注入相关上下文。其组件包括本地 HTTP worker 和网页查看器、SQLite 数据库、Chroma 向量数据库，以及按搜索、时间线、观察详情分层获取信息的 MCP 工具。README 表示可接入 Claude Code、OpenClaw、Codex、Gemini、Copilot、OpenCode 等工具，并说明另有可选云同步服务。最低环境要求包括 Node.js 20、Bun、uv 等；许可证文件和 API 均为 Apache-2.0。默认分支 main 的采集提交为 d8bf121ae98063e5e3158befff33a9d635c0246b。 当前可支持的用途： 按项目文档，可在受支持的代理会话中保留代码库任务观察和摘要，并通过 MCP 搜索先取简要索引，再按需取更完整细节。文档未能证明跨所有代理版本的行为一致。 潜在价值（分析推测）： 对长期维护代码库的个人或团队，持久化的项目上下文可能减少重新解释背景的时间；效果取决于记录准确性、检索召回率、敏感信息过滤、存储管理和代理集成稳定性。将上下文发送至模型或启用云同步会增加数据处理边界，必须先审查隐私、保留期限和访问控制。 对象、入门与成本： 面向频繁使用编码代理并希望跨会话保留上下文的开发者。建议从非敏感测试仓库开始，审查钩子、配置项、数据库位置和网络请求，再逐步开放项目。主要成本是本地存储和向量检索资源、安装运行时、模型 API 用量，以及可选云同步费用；各服务价格取决于供应商和调用量，本文未估算金额。 复刻：qinshanliangengineer/claude-mem，fork parent 已核实为 thedotmack/claude-mem；fork main 落后源仓库 675 个提交，本次未同步。

来源： 官方 README、LICENSE、GitHub 仓库元数据、采集时源分支提交。

采集与归档说明

- 实时榜单证据已保存为同目录 trending.html；采集页为 GitHub Trending 当日榜单，不使用缓存榜单或全站总星标排序替代。 - 五个仓库的 README、仓库 API 元数据、LICENSE（worktrunk 使用 LICENSE 中的双许可证说明）及默认分支 SHA 已核验。API 星标快照分别为 5,096、11,053、313,353、29,648、112,880；源 SHA 与采集内容一同记录在 manifest.json。 - 五个账号内同名仓库均为真实 fork，parent 与榜单源仓库匹配。为避免覆盖用户修改，本次不执行同步；worktrunk 落后 5 个提交、skills 落后 495 个提交、claude-mem 落后 675 个提交，auto-ontology 和 floci 与上游一致。 - GitHub 汇总仓库 qinshanliangengineer/QSL-new-projects 已确认是公开仓库，写入前 main 分支头为 3eedfe6f67e28dfce63cddbb50bccbbd5201ff84，当日目录此前不存在。远端归档结果将在提交后核验；本地文件生成不等于远端已上传。